

KnowledgeBase

```

ABOR_EXPANDED : bool
CLASSIFIED : bool
KB_LOADED : bool
KB_UNSAT : bool

abstract_roles : set[str]
acyclic_box : bool
applied_trans_rules : list[str]
assertions : list[Assertion]
atomic_concepts : dict[str, Concept]
axioms_A_equiv_C : dict[str, set[Concept]]
axioms_A_is_a_B : dict[str, set[PrimitiveConceptDefinition]]
axioms_A_is_a_C : dict[str, set[PrimitiveConceptDefinition]]
axioms_C_equiv_D : list[ConceptInclusion]
axioms_C_is_a_A : dict[str, set[GeneralConceptInclusion]]
axioms_C_is_a_D : dict[str, set[GeneralConceptInclusion]]
axioms_to_do_A_is_a_C : dict[str, set[PrimitiveConceptDefinition]]
axioms_to_do_A_is_a_B : dict[str, set[PrimitiveConceptDefinition]]
axioms_to_do_C_is_a_A : dict[str, set[GeneralConceptInclusion]]
axioms_to_do_C_is_a_D : dict[str, set[GeneralConceptInclusion]]
axioms_to_do_imp_A_is_a_C : dict[str, set[PrimitiveConceptDefinition]]
axioms_to_do_imp_C_is_a_A : dict[str, set[GeneralConceptInclusion]]
axioms_to_do_imp_C_is_a_D : dict[str, set[GeneralConceptInclusion]]
blocked_assertions : dict[str, list[Assertion]]
blocked_axiom_assertions : dict[str, list[Assertion]]
blocking_dynamic : bool
blocking_type

concept_individual_list : dict[int, SortedSet[CreatedIndividual]]
concrete_concepts : dict[str, FuzzyConcreteConcept]
concrete_features : dict[str, ConcreteFeature]
concrete_fuzzy_concepts : bool
concrete_roles : set[str]
directly_blocked_children : dict[str, list[str]]
disjoint_variables : dict[str, set[str]]
domain_restrictions : dict[str, set[Concept]]
exist_assertions : list[Assertion]
functional_roles : set[str]
fuzzy_numbers : dict[str, TriangularFuzzyNumber]
individuals : dict[str, Individual]
inverse_functional_roles : set[str]
inverse_roles : dict[str, set[str]]
labels_with_nodes : dict[str, set[str]]
language : str
lazy_unfoldable : bool
lazy_unfoldable : bool
max_depth : int
mlip
modifiers : dict[str, Modifier]
nodes_classification : list[ClassificationNode]
num_assertions : int
num_defined_concepts : int
num_defined_individuals : int
num_relations : int
number_of_concepts : dict[str, int]
number_of_roles : dict[str, int]
old_01_variables : int
old_binary_variables : int
order : dict[str, int]
positive_concrete_value_assertions : list[Assertion]
processed_assertions : set[int]
r_successors : dict[str, list[str]]
range_restrictions : dict[str, set[Concept]]
reflexive_roles : set[str]
roles_with_all_parents : dict[str, dict[str, float]]
roles_with_parents : dict[str, dict[str, float]]
roles_with_trans_children : dict[str, list[str]]
rule_acyclic_box : bool
rules_applied : dict[KnowledgeBaseRules, int]
show_language : bool
similarity_relations : set[str]
subsumption_flags : dict[str, dict[str, float]]
symmetric_roles : set[str]
t_G : list[GeneralConceptInclusion]
t_definitions : dict[str, Concept]
t_disjoints : dict[str, set[str]]
t_inclusions : dict[str, set[PrimitiveConceptDefinition]]
t_synonyms : dict[str, set[str]]
temp_relations_list : dict[str, list[Relation]]
temp_string_concept_list : NoneType, list[Concept]
temp_string_list : NoneType, list[str]
imp_features : list[str]
relative_roles : set[str]
truth_constants : dict[str, float]
x_prime_individuals : dict[str, list[str]]
y_prime_individuals : dict[str, list[str]]

```

```

__add_assertion_1(new_assertion: Assertion): None
__add_assertion_2(a: Individual, c: Concept, n: Degree): None
__add_assertion_3(a: Individual, restrict: Restriction): None
__add_concepts_disjoint_1(disjoint_concepts: list[Concept]): None
__add_concepts_disjoint_2(c1: str, c2: str): None
__add_concepts_disjoint_3(c: Concept, d: Concept): None
__add_criap_concrete_concept_equations(concept: CriapConcreteConcept, x_c: Variable, x_assert: Variable): None
__add_fuzzy_concrete_concept_equation(concept: FuzzyConcreteConcept, x_c: Variable, x_assert: Variable): None
__add_left_concrete_concept_equations(concept: LeftConcreteConcept, x_c: Variable, x_assert: Variable): None
__add_linear_concrete_concept_equations(concept: LinearConcreteConcept, x_A_is_C: Variable, x_assert: Variable): None
__add_right_concrete_concept_equations(concept: RightConcreteConcept, x_c: Variable, x_assert: Variable): None
__add_sigma_count_equation(sigma: Variable, xs_in_C: list[Variable]): None
__add_trapezoidal_concrete_concept_equations(concept: TrapezoidalConcreteConcept, x_c: Variable, x_assert: Variable): None
__add_triangular_concrete_concept_equations(concept: TriangularConcreteConcept, x_c: Variable, x_assert: Variable): None
concept_absorption_1(pod: PrimitiveConceptDefinition, atomic: bool): bool
concept_absorption_2(aas: GeneralConceptInclusion, atomic: bool): bool
degree_if_not_one_1(deg: Degree): str
degree_if_not_one_2(d: float): str
gc_transformation_1(aas: GeneralConceptInclusion, atomic: bool): bool
gc_transformation_2(pod: PrimitiveConceptDefinition): bool
get_correct_version_of_individual_1(aas: Assertion): None
get_correct_version_of_individual_2(aas: Relation): None
get_new_individual_1(): CreatedIndividual
get_new_individual_2(parent: typing.Optional[Individual], f_name: typing.Optional[str]): CreatedIndividual
__init__(): None
remove_A_is_a_X_1(key: str, pod: PrimitiveConceptDefinition, pod_dict: dict[str, set[PrimitiveConceptDefinition]]): None
remove_A_is_a_X_2(key: str, pod: PrimitiveConceptDefinition, atomic: bool): None
restrict_range_1(x_b: Variable, k1: float, k2: float): None
restrict_range_2(x_b: Variable, x_f: Variable, k1: float, k2: float): None
role_absorption_1(aas: PrimitiveConceptDefinition): bool
role_absorption_2(aas: GeneralConceptInclusion, atomic: bool): bool
role_implies_1(subsumer: str, subsumed: str): None
role_implies_2(subsumer: str, subsumed: str, n: float): None
role_subsumes_bool_1(subsumer: str, subsumed: str, n: float): bool
role_subsumes_bool_2(subsumer: str, subsumed: str, n: float, p_list: dict[str, dict[str, float]]): bool
solve_cardinality_sigma: Variable, n: Individual, C: list[Individual], n: str, C: Concept): None
solve_gc_1(ind: Individual, gc: GeneralConceptInclusion): None
solve_gc_2(ind: Individual): None
solve_reflexive_roles_1(ind: Individual): None
solve_reflexive_roles_2(): None
solve_role_inclusion_axioms_1(): None
solve_role_inclusion_axioms_2(ind: Individual, n: Relation): None
add_assertion(new_assertion: Assertion): None
add_assertions(list_of_assertions: list[Assertion]): None
add_atomic_concepts_disjoint(disjoint_concepts: list[str]): None
add_axiom_to_A_equiv_C(a: str, conc: Concept): None
add_axiom_to_A_is_a_C(a: str, pod: PrimitiveConceptDefinition, pod_dict: dict[str, set[PrimitiveConceptDefinition]]): None
add_axiom_to_C_is_a_A(conc1: Concept, conc2: Concept, degree: Degree, logic_type: LogicOperatorType): None
add_axiom_to_C_is_a_D(conc1: Concept, conc2: Concept, degree: Degree, logic_type: LogicOperatorType): None
add_axiom_to_C_is_a_X(conc1: Concept, conc2: Concept, degree: Degree, logic_type: LogicOperatorType): None
add_axiom_to_do_A_is_a_X(a: str, pod: PrimitiveConceptDefinition): None
add_axiom_to_incls(a: str, pod: PrimitiveConceptDefinition): None
add_axioms_to_1g(): None
add_concept(concept_name: str, conc: FuzzyConcreteConcept): None
add_concepts_disjoint(disjoint_concepts: list[str]): None
add_created_individual(ind_name: str, ind: CreatedIndividual): None
add_datatype_restriction(restriction_type: RestrictionType, c: typing.Any, f_name: str): Concept
add_disjoint_union_concepts(disjoint_union_concepts: list[str]): None
add_equivalence_relation(role: str): None
add_equivalent_concepts(equiv_concepts: list[Concept]): None
add_equivalent_roles(equiv_roles: list[str]): None
add_fuzzy_number(f_name: str, f: TriangularFuzzyNumber): None
add_gc(conc1: Concept, conc2: Concept, degree: Degree, logic_type: LogicOperatorType): None
add_individual(ind_name: str, ind: Individual): None
add_individual_to_concept(concept_id: int, ind: Individual): None
add_inverse_roles(role: str, inv_role: str): None
add_labels_with_nodes(node: str, ind_name: str): None
add_modifier(mod_name: str, mod: Modifier): None
add_mutually_disjoint(c1: str, c2: str): None
add_negated_datatype_restriction(aas: Assertion): None
add_negated_equations(individual: Individual, c: Concept): None
add_parent_recursively(role_c: str, all_parents: dict[str, float], current_role: str, n1: float): None
add_relation(ind_A: Individual, role: str, ind_B: Individual, degree: Degree): Relation
add_relation_with_role_parent(ind: Individual, role_c: str, role_p: str, n: float): None
add_relation_with_role_parent_in_lukasiewicz(Relation, role_p: str, n: float): None
add_similarity_relation(role: str): None
add_simple_inverse_roles(role: str, inv_role: str): None
add_subsumptions(conc2: Concept, conc1: Concept, degree: Degree, logic_type: LogicOperatorType): None
add_tset_links(DiGraph, A, f_C: dict[str, int], use_idr: bool): bool
add_tr_links(DiGraph, A, f_C: dict[str, int], used_roles: set[str], v: int): bool
add_trc_links(DiGraph, A, f_C: dict[str, int], use_idr: bool): bool
add_imp_feature(feature: str): None
check_fuzzy_number_concept_exists(conc_name: str): bool
check_individual_exists(ind_name: str): bool
check_role(role_name: str, conc: Concept): None
check_trans_role_applied(rel: Relation, restrict: Restriction): bool
clone(): typing.Self
clone_without_abox(): typing.Self
compute_blocking_type(): None
compute_language(): None
compute_variables_old_calculus(for: FuzzyConcreteConcept): None
concept_absorption(pod: PrimitiveConceptDefinition, atomic: bool): bool
concept_exists(name: str): bool
convert_strings_into_integers(): None
create_roles_with_all_parents(): None
create_roles_with_trans_children(): None
define_atomic_concept(concept_name: str, conc: Concept, implication: LogicOperatorType, n: float): None
define_boolean_concrete_feature(fun_role: str): None
define_concept(concept_name: str, conc: Concept): None
define_concrete_feature(role: str): None
define_equivalent_concepts(c1: Concept, c2: Concept): None
define_integer_concrete_feature(fun_role: str, d1: int, d2: int): None
define_real_concrete_feature(fun_role: str, d1: float, d2: float): None
define_string_concrete_feature(fun_role: str): None
define_synonym(concept_name_1: str, concept_name_2: str): None
define_synonym_concept(name_1: str, concept_name_2: str): None
definition_absorption_1(a, do): pod: PrimitiveConceptDefinition): bool
degree_if_not_one(deg: Degree): str
disjoint_with_defined_concepts(str): bool
exists_primitive_concept_definition(pods: set[PrimitiveConceptDefinition], pod: PrimitiveConceptDefinition): bool
exit_condition(): None
exit_condition_A_is_a_X(pod: PrimitiveConceptDefinition): None
exit_condition_A_is_a_X(pod: GeneralConceptInclusion): None
form_inv_role_inc_axioms(): None
form_inv_role_relations(): None
form_inv_trans_roles(): None
gc_transform_defina_atomic_concept(concept_name: str, conc: Concept, implication: LogicOperatorType, n: float)
gc_transformation(aas: GeneralConceptInclusion, atomic: bool): bool
gc_transformation_add_axiom_to_C_is_a_X(conc1: Concept, conc2: Concept, degree: Degree, logic_type: LogicOperatorType): None
gc_transformations_A_is_a_C(): None
gc_transformations_C_is_a_A(): None
gc_transformations_C_is_a_D(): None
get_A_f_C(): dict[str, int]
get_classification_node(): typing.Optional[ClassificationNode]
get_concept(name: str): Concept
get_concept_from_numbers(n: int): typing.Optional[str]
get_correct_version_of_individual(aas: Assertion): None
get_inclusion_degree(subsumer: str, subsumed: str): float
get_individual(ind_name: str) typing.Union[Individual, CreatedIndividual]
get_individuals(): dict[str, Individual]
get_inverses_of_inverse_role(role: str): typing.Optional[set[str]]
get_language(): str
get_logic(): FuzzyLogic
get_named_individuals(): list[Individual]
get_new_atomic_concept(): Concept
get_new_concrete_individual(parent: Individual, f_name: str): CreatedIndividual
get_new_individual(): CreatedIndividual
get_new_individual_common_code(parent: typing.Optional[Individual], f_name: typing.Optional[str]): CreatedIndividual
get_number_from_concept(concept_name: str): int
get_number_of_domain_restrictions(): int
get_number_of_range_restrictions(): int
get_subsumption_flags(): ClassificationNode: float
get_imp_feature(feature: str): str
get_truth_constants(): str: typing.Optional[float]
goedel_implies(conc1: Concept, conc2: Concept, degree: Degree): None
has_functional_abstract_roles(): bool
has_nominals_in_abox(): bool
has_nominals_in_box(): bool
has_only_criap_rule_concepts(): Concept: bool
implies(conc1: Concept, conc2: Concept, degree: Degree): None
is_assertion_processed(aas: Assertion): bool
is_atomic_criap_concept(c: Concept): bool
is_classified(): bool
is_concrete_type(c: Concept): bool
is_criap_concept(concept_name: str): bool
is_criap_role(role_name: str): bool
is_lazy_unfoldable(): bool
is_loaded(): bool
is_redundant_A_is_a_C(concept_name: str, conc: Concept, implication: LogicOperatorType, n: float): bool
is_redundant_gc(C: Concept, D: Concept, implication: LogicOperatorType, n: float): bool
is_box_acyclic(): bool
kleans_dienes_implies(conc1: Concept, conc2: Concept, degree: Degree): None
lukasiewicz_implies(conc1: Concept, conc2: Concept, degree: Degree): None
mark_process_assertion(aas: Assertion): None
merge(): Individual, B: Individual): None
merge_fillers(ind: Individual, func: str): None
nominal_absorption(conc1: Concept, conc2: Concept, degree: Degree): bool
optimize(Expression): Solution
partition_loop_A_is_a_B(): None
partition_loop_A_is_a_C(): None
partition_loop_C_is_a_A(): None
partition_loop_C_is_a_D(): None
partition_loop_to_do_A_is_a_B(): None
partition_loop_to_do_A_is_a_C(): None
preprocess_box(): None
print_box(): None
read_object_from_file(file_path: str): KnowledgeBase
remove_A_is_a_B(key: str, pod: PrimitiveConceptDefinition): None
remove_A_is_a_C(key: str, pod: PrimitiveConceptDefinition): None
remove_A_is_a_X(key: str, pod: PrimitiveConceptDefinition, pod_dict: dict[str, set[PrimitiveConceptDefinition]]): None
remove_C_is_a_A(key: str, gc: GeneralConceptInclusion): None
remove_C_is_a_D(key: str, gc: GeneralConceptInclusion): None
remove_C_is_a_X(key: str, gc: GeneralConceptInclusion, atomic: bool): None
represent_box_with_gcls(): None
restrict_range(x_b: Variable, k1: float, k2: float): None
role_absorption(aas: PrimitiveConceptDefinition): bool
role_domain(role_a: str, conc: Concept): None
role_implies(subsumer: str, subsumed: str): None
role_is_functional(role: str): None
role_is_inverse_functional(role: str): None
role_is_reflexive(role: str): None
role_is_symmetric(role: str): None
role_is_transitive(role: str): None
role_range(role: str, conc: Concept): None
role_subsumes_bool(subsumer: str, subsumed: str, n: float): None
role_subsumes_bool(subsumer: str, subsumed: str, n: float): bool
rule_and(aas: Assertion): None
rule_and_ax(a: Individual, c: Concept, v: str): None
rule_atomic(aas: Assertion): None
rule_bottom(aas: Assertion): None
rule_choque(aas: Assertion): None
rule_complemented(): Individual, c: Concept): None
rule_complemented_at_least_datatype_restriction(): CreatedIndividual, aas: Assertion): None
rule_complemented_extended_negative_threshold(aas: Assertion): None
rule_complemented_choquet(aas: Assertion): None
rule_complemented_complex_assertion(aas: Assertion): None
rule_complemented_conjunct(aas: Assertion): None
rule_complemented_exact_datatype_restriction(): CreatedIndividual, aas: Assertion): None
rule_complemented_extended_negative_threshold(aas: Assertion): None
rule_complemented_fuzzy_number(aas: Assertion): None
rule_complemented_goedel_implication(aas: Assertion): None
rule_complemented_has_value(aas: Assertion): None
rule_complemented_lazy_unfolding(aas: Assertion): None
rule_complemented_modified(aas: Assertion): None
rule_complemented_negative_threshold(aas: Assertion): None
rule_complemented_or(aas: Assertion): None
rule_complemented_positive_threshold(aas: Assertion): None
rule_complemented_quantified_existence(aas: Assertion): None
rule_complemented_qual_superox(aas: Assertion): None
rule_complemented_self(aas: Assertion): None
rule_complemented_sigma_concept(aas: Assertion): None
rule_complemented_weighted_concept(aas: Assertion): None
rule_complemented_weighted_max(aas: Assertion): None
rule_complemented_weighted_min(aas: Assertion): None
rule_complemented_weighted_sum(aas: Assertion): None
rule_complemented_weighted_sum_pero(aas: Assertion): None
rule_complemented_zadeh_implication(aas: Assertion): None
rule_concrete(aas: Assertion): None
rule_domain_lazy_unfolding(domain_role: str, rel: Relation): None
rule_extended_negative_threshold(domain_aas: Assertion): None
rule_extended_positive_threshold(aas: Assertion): None
rule_fuzzy_number(aas: Assertion): None
rule_goedel_and(aas: Assertion): None
rule_goedel_implication(aas: Assertion): None
rule_goedel_or(aas: Assertion): None
rule_has_value(aas: Assertion): None
rule_lazy_unfolding(aas: Assertion): None
rule_lower_approximation(aas: Assertion): None
rule_lower_upper_approximation(aas: Assertion): None
rule_lukasiewicz_and(aas: Assertion): None
rule_lukasiewicz_or(aas: Assertion): None
rule_modified(aas: Assertion): None
rule_n1(): None
rule_n2(): None
rule_negative_threshold(aas: Assertion): None
rule_or(aas: Assertion): None
rule_ova(aas: Assertion): None
rule_positive_threshold(aas: Assertion): None
rule_quantified_ova(aas: Assertion): None
rule_qual_superox(aas: Assertion): None
rule_range_lazy_unfolding(range_role: str, rel: Relation): None
rule_sigma_concept(aas: Assertion): None
rule_some(aas: Assertion): None
rule_sigma(aas: Assertion): None
rule_threshold_common(x_a_in_c: Variable, x_a_in_c: Variable, y: Variable): None
rule_right_lower_approximation(aas: Assertion): None
rule_top(aas: Assertion): None
rule_upper_approximation(aas: Assertion): None
rule_weighted_concept(aas: Assertion): None
rule_weighted_max(aas: Assertion): None
rule_weighted_min(aas: Assertion): None
rule_weighted_sum(aas: Assertion): None
rule_weighted_sum_pero(aas: Assertion): None
rule_zadeh_implication(aas: Assertion): None
save_absorbed_box_to_file(output: typing.Callable): None
save_box_common_part_to_file(output: typing.Callable): None
save_box_to_file(output: typing.Callable): None
save_to_file(file_name: str): None
set_criap_concept(c: Concept): None
set_criap_role(role_name: str): None
set_dynamic_blocking(): None
set_logic(logic: FuzzyLogic): None
set_truth_constants(): str, w: float): None
set_unsatisfiable(KB): None
show_statistics(): None
solve_abox(): None
solve_assertions(): None
solve_cardinality_list(): None
solve_choquet_integral_assertion(ind: Individual, c: ChoquetIntegral): None
solve_choquet_integral_complemented_assertion(ind: Individual, c: OperatorConcept): None
solve_concept_absorption(ind: Individual, concept: Concept): None
solve_concept_complemented_assertion(ind: Individual, lower_limit: Degree, concept: Concept): None
solve_concrete_value_assertions(): None
solve_criap_concrete_concept_assertion(ind: Individual, concept: CriapConcreteConcept): None
solve_domain_and_range_axioms(): None
solve_functional_roles(): None
solve_fuzzy_concrete_concept_complement_assertion(ind: CreatedIndividual, lower_limit: Degree, cur_concept: OperatorConcept): None
solve_gc(ind: Individual, gc: GeneralConceptInclusion): None
solve_inverse_roles(): None
solve_kb(): None
solve_kleane_dienes_gc(ind: Individual, gc: GeneralConceptInclusion): None
solve_left_concrete_concept_assertion(ind: CreatedIndividual, concept: LeftConcreteConcept): None
solve_linear_concrete_concept_assertion(ind: CreatedIndividual, concept: LinearConcreteConcept): None
solve_linear_modifier_assertion(ind: Individual, con: Concept, modifier: LinearModifier): None
solve_lukasiewicz_gc(ind: Individual, gc: GeneralConceptInclusion): None
solve_modifier_assertion(ind: Individual, concept: Concept, modifier: Modifier): None
solve_modifier_complemented_assertion(ind: Individual, concept: OperatorConcept, degree: Degree): None
solve_one_exist_assertion(): None
solve_ova_assertion(ind: Individual, c: typing.Union[ChoConcept, CowsConcept]): None
solve_ova_complemented_assertion(ind: Individual, cur_concept: OperatorConcept): None
solve_reflexive_role(role: str): None
solve_reflexive_roles(ind: Individual): None
solve_right_concrete_concept_assertion(ind: CreatedIndividual, concept: RightConcreteConcept): None
solve_role_inclusion_axioms(): None
solve_sigma_integral_assertion(ind: Individual, concept: typing.Union[QuantifierIntegral, QuantifierIntegral]): None
solve_synonym_integral_complemented_assertion(ind: Individual, cur_concept: OperatorConcept): None
solve_trapezoidal_concrete_concept_assertion(ind: CreatedIndividual, concept: TrapezoidalConcreteConcept): None
solve_triangular_concrete_concept_assertion(ind: CreatedIndividual, concept: TriangularConcreteConcept): None
solve_triangular_modifier_assertion(ind: Individual, concept: Concept, modifier: TriangularModifier): None
solve_w_max_complemented_assertion(ind: Individual, cur_concept: OperatorConcept): None
solve_w_min_complemented_assertion(ind: Individual, concept: WeightedMinConcept): None
solve_w_min_complemented_assertion(ind: Individual, cur_concept: OperatorConcept): None
solve_w_sum_assertion(ind: Individual, concept: WeightedSumConcept): None
solve_w_sum_complemented_assertion(ind: Individual, cur_concept: OperatorConcept): None
solve_w_sum_zero_complemented_assertion(ind: Individual, cur_concept: OperatorConcept): None
solve_zadeh_gc(ind: Individual, gc: GeneralConceptInclusion): None
synonym_absorption_A_is_a_B(pod1: PrimitiveConceptDefinition): bool
synonym_absorption_A_is_a_B(pod2: PrimitiveConceptDefinition): bool
unlock_children(parent: str): None
unlock_individual(ind_name: str): None
write_object_to_file(file_path: str): None
zadeh_implies(conc1: Concept, conc2: Concept): None

```